

DA3408 - AI-OPS Lab

MOd3 Assignment

Vangala VedaVachan Reddy DA24B029
August 2026

Question 1: SingleStage vs. Multi-Stage Docker

1. Naive Single Stage Docker Build

I first created a single-stage Dockerfile using the `python:3.11` base image and the required dependencies were installed using the `requirements.txt` file and the FastAPI application along with the trained `model.joblib` was copied into the image.

The resulting image size was:

Naive image size: 615 MB

I ran the image as a container and tested the API. The health check returned `status: ok`.

```
/healthz -> {"status":"ok"}
Spam      -> {"label":"spam"}
Ham       -> {"label":"ham"}
```

2. Multi-Stage Docker Build

Now I created a multi-stage Dockerfile. The builder stage uses `python:3.11` to install the required dependencies and the final stage uses the smaller `python:3.11-slim` image and copies only the installed dependencies, application code and trained model.

The resulting image size was:

Multi-stage image size: 165 MB

The percentage reduction in image size is:

$$\frac{615 - 165}{615} \times 100 = \mathbf{73.17\%}$$

The multistage container was also tested with the same API endpoints and

```
/healthz -> {"status":"ok"}
Spam      -> {"label":"spam"}
Ham       -> {"label":"ham"}
```

3. Why the Multi-Stage Image is Smaller

The naive image uses the full `python:3.11` image while the final stage uses the smaller `python:3.11-slim` image. The builder stages extra base-image tooling and pip installation or cache contents are not copied to the final image only the required Python packages `app.py` and `model.joblib` are included.

Question 2: Multi-Container Orchestration with Docker Compose

3. Cache Hit Speedup

I sent the same prediction request twice and measured the execution time using the `time` command. The first request was a cache miss and took 0.133 seconds. The second identical request was served from the cache and took 0.027 seconds.

Request	Time
First request (Cache Miss)	0.133 s
Second request (Cache Hit)	0.027 s

The second request was approximately 4.9 times faster providing measurable evidence that the Redis cache improved the response time.

4. Docker Compose vs. Kubernetes

1-Docker Compose helps run multiple related containers together on a single machine and it manages the services, networking and application stack while Kubernetes manages containerized applications across a cluster of machines.

2-Docker Compose is mainly used for singlehost multi-container orchestration whereas Kubernetes provides cluster level management, scaling and recovery.

Question 3: Kubernetes Indexed Job – Parallel Data Validation

1. Indexed Job Design and Parallelism

I generated eight CSV shards, with 50 rows in each shard. The cluster was configured with 2 nodes, each having 2 allocatable CPUs, giving 4 CPUs in total. Since each validation pod requests and is limited to 1 CPU, I selected `parallelism: 4`. Therefore, four shards can be processed concurrently, with 8 total completions.

The Job uses `completionMode: Indexed`, so each pod receives a unique `JOB_COMPLETION_INDEX`. The Downward API provides the pod name using `metadata.name` and the node name using `spec.nodeName`. CPU requests and limits are both set to 1 CPU, and `restartPolicy: Never` is used.

2. Actual Parallelism Evidence

The Job was applied successfully and four pods were observed running concurrently. The pods were distributed across the two nodes, with two pods on each node, confirming that the selected parallelism of 4 was actually reached.

```
shard-validator-0    Running    minikube-m02
shard-validator-1    Running    minikube-m02
shard-validator-2    Running    minikube
shard-validator-3    Running    minikube
```

The final Job completed successfully with:

```
shard-validator    Complete    8/8
```

3. Validation Results

I collected the pod logs using the Kubernetes Python API through `read_namespaced_pod_log()`, rather than using a shared volume.

Shard	Rows	Invalid Rows
0	50	3
1	50	5
2	50	2
3	50	7
4	50	1
5	50	4
6	50	6
7	50	0
Total	400	28

All 8 shards were reported successfully. The shard CSV files were included in the container image instead of using a shared volume. This avoids depending on shared storage between nodes and is suitable for this workload because every pod has access to the required shard data.

4. If 6 CPUs Were Available

If the cluster instead had 3 nodes with 2 CPUs each, there would be 6 CPUs available. Since each validation pod uses 1 CPU, I would increase `parallelism` to 6 so that six shards could be processed concurrently. The remaining two shards would then run after the first six complete.

Question 4: Kubernetes Deployment – Self-Healing and Rolling Updates

2. Self-Healing

I manually deleted the running pod `spam-api-78ddd4498d-2b9kt`. Kubernetes automatically created a replacement pod, `spam-api-78ddd4498d-sgs8q`, while the other replica continued running.

The ReplicaSet controller managed by the Deployment maintains the desired number of replicas. When it detects that the number of running pods is below the desired count of two, it creates a replacement pod automatically.

3. Rolling Update

I made a visible change to the API by adding a version field to the `/healthz` response and i built the updated application as `spam-api:v2` and updated the Deployment to use the new image.

```
deployment "spam-api" successfully rolled out
```

The updated API returned:

```
{"status":"ok","version":"v2"}
```

The rollout history showed three Deployment revisions:

REVISION	CHANGE-CAUSE
1	<none>
2	<none>
3	<none>

If the application had been deployed as a Job instead of a Deployment it would have been treated as a run to completion workload rather than a continuously running service. A Job would therefore not provide the same replica maintenance and rolling update behavior required by this API.

4. Deployment vs. Job

A Deployment is suitable for the spam-detection API because it maintains long-running replic and replaces failed pods and supports rolling updates. A Job is suitable for the shard validation workload from Question 3 because each pod performs a finite validation task and the workload completes after all shards have been processed.